HfT Stuttgart
Fakultät C

# Interdisziplinäres Projekt Flexible Roboterprogrammierung für Montage und Demontage

# **Abschlussbericht**

Prof. Dr.-Ing. Darko Katic

**Valentin Bäuerle**

21bava1bif@hft-stuttgart.de
Informatik
Matr.-Nr. 1004286

# Eidesstattliche Erklärung

Hiermit erkläre ich Valentin Bäuerle (1004286), dass ich den vorliegenden Abschlussbericht selbstständig und ohne fremde Hilfe bzw. unerlaubte Hilfsmittel angefertigt, andere als die angegebenen Quellen und Hilfsmittel nicht benutzt und die den benutzten Quellen wörtlich oder inhaltlich entnommenen Stellen als solche kenntlich gemacht habe.

# Sperrvermerk

Die vorliegende Arbeit beinhaltet interne und vertrauliche Informationen der HFT Stuttgart. Valentin Bäuerle verpflichtet sich, alle im Rahmen der Arbeit gewonnenen Informationen und Erkenntnisse für einen Zeitraum von 5 Jahren vertraulich zu behandeln, d.h. sie nicht Dritten zugänglich zu machen oder anderweitig zu verwerten. Dies gilt auch für Familienangehörige oder Personen, die bei der Erstellung der Arbeit – sei es durch Korrekturlesen oder ähnliches - behilflich sind. Die Arbeit darf den betreuenden Professoren ausschließlich zur fachlichen Bewertung vorgelegt werden. Die Weitergabe des Inhalts der Arbeit im Gesamten oder in Teilen sowie das Anfertigen von Kopien oder Abschriften – auch in digitaler Form – sind grundsätzlich untersagt. Publikationen jeglicher Art (gilt auch für die Aufnahme im hochschuleigenen Bibliotheksbestand) bedürfen der schriftlichen Genehmigung der HFT Stuttgart.

Waiblingen, den 15.05.2026



.....
Unterschrift (Valentin Bäuerle)

# Inhaltsverzeichnis

**1 Einleitung** **1**
1.1 Einstieg . . . . . 1
1.2 Motivation und Problemstellung . . . . . 1
1.3 Stand der Technik . . . . . 2
1.4 Aufbau des Berichts . . . . . 3

**2 Grundlagen und theoretischer Rahmen** **4**
2.1 Der Dobot Magician als Plattform . . . . . 4
2.1.1 Grundaufbau . . . . . 4
2.1.2 Bewegung & Präzision . . . . . 4
2.1.3 Nutzlast & Dynamik . . . . . 5
2.1.4 Schnittstellen & Kommunikation . . . . . 5
2.1.5 Betrieb & Umgebung . . . . . 5
2.2 Entwicklungs- und Simulationsumgebung . . . . . 5
2.2.1 Virtuelle Maschine & Betriebssystem . . . . . 5
2.2.2 ROS2 & RViz . . . . . 6
2.2.3 MuJoCo als Lernumgebung . . . . . 6
2.2.4 Konzept des Sim-to-Real-Transfer . . . . . 7
2.3 Definition von Flexibilität im Kontext dieser Arbeit . . . . . 8
2.4 Montage und Demontage als Robotikproblem . . . . . 9

**3 Fazit und begründete Empfehlung** **12**

**Literaturverzeichnis** **13**

# 1 Einleitung

## 1.1 Einstieg

Das manuelle Montieren und Demontieren von Bauteilen ist ein zentrales Element moderner Fertigungsprozesse, insbesondere bei der Montage von Elektronikkomponenten, der Automobilproduktion und der Luft- und Raumfahrt [1]. Da manuelle Montageprozesse zeit- und kostenintensiv sind, ist deren Automatisierung ein zentrales Ziel in der Robotik [2], um Effizienz, Präzision und Sicherheit zu erhöhen. Roboterarme werden zunehmend eingesetzt, um diese Aufgaben zu übernehmen[3].

Hierbei besteht die Herausforderung darin, dass Bauteile fertigungsbedingt stets Lage- und Orientierungstoleranzen aufweisen [4]. Selbst geringe Lageabweichungen können dazu führen, dass klassische, deterministische Programme versagen, da sie keine Mechanismen zur Lagekorrektur oder Fehlertoleranz besitzen [4].

Diese Einschränkungen motivieren den Einsatz lernbasierter Verfahren wie Reinforcement Learning oder Imitation Learning, die sich an variierende Umgebungsbedingungen anpassen und vorgegebene Aufgaben dennoch zuverlässig ausführen können [5]. Da reale Experimente aufwendig und fehleranfällig sind, werden solche Verfahren häufig zunächst in physikbasierten Simulationsumgebungen entwickelt und evaluiert. Im Folgenden wird die daraus resultierende Problemstellung dieser Arbeit näher erläutert.

## 1.2 Motivation und Problemstellung

Diese Vorarbeit verfolgt einen explorativen Ansatz: Anstatt einen einzelnen Algorithmus zu implementieren und zu evaluieren, werden bestehende Verfahren zur roboterbasierten Montage und Demontage systematisch untersucht und verglichen.

Die zentrale technische Herausforderung liegt darin, dass klassische Steuerungsansätze an den unvermeidlichen Lage- und Orientierungstoleranzen realer Bauteile scheitern [4], was den Einsatz adaptiverer Verfahren motiviert.

Eine zusätzliche, strukturell eigenständige Herausforderung ergibt sich aus dem sogenannten Sim-to-Real-Gap: Verfahren, die in der Simulation erfolgreich trainiert oder erprobt werden, lassen sich häufig nicht direkt auf reale Systeme übertragen [6]. Reibung, Sensorungenauigkeiten und mechanische Toleranzen erzeugen eine Diskrepanz zwischen Simulations- und Realverhalten [6], die eine gesonderte Behandlung erfordert. Lernbasierte Verfahren, insbesondere Reinforcement Learning und Imitation Learning, versprechen, beide Herausforderungen zu adressieren: Sie können sich an variierende Umgebungsbedingungen anpassen und prinzipiell in der Simulation trainiertes Verhalten auf reale Systeme übertragen [5]. Allerdings gehen sie mit erheblichem Implementierungsaufwand und hohem Datenbedarf einher [5].

Um eine fundierte Entscheidung für die anschließende Bachelorarbeit treffen zu können, ist ein systematischer Vergleich der verfügbaren Ansätze anhand klar definierter Kriterien notwendig, darunter Adaptivität gegenüber Lageabweichungen, Implementierungsaufwand sowie Sim-to-Real-Fähigkeit. Ziel dieser Vorarbeit ist es, bestehende Ansätze systematisch zu vergleichen und denjenigen zu identifizieren, der sich am besten für den Einsatz an einem realen Roboterarm und eine vertiefte Untersuchung im

Rahmen der Bachelorarbeit eignet.

## 1.3 Stand der Technik

Es existiert ein breites Spektrum an Verfahren zur roboterbasierten Montage und Demontage. Diese reichen von klassischen, planungsbasierten Ansätzen über lernbasierte Methoden bis hin zu neueren, modellbasierten Großarchitekturen. Alle Ansätze sind aktiver Forschungsgegenstand und werden kontinuierlich weiterentwickelt [7, vgl. Kap. 2].

Klassische Bewegungsplanungsverfahren, etwa auf Basis von RRT oder PRM, erzeugen kollisionsfreie Trajektorien in bekannten Umgebungen zuverlässig und nachvollziehbar [8]. Ihre grundlegende Einschränkung besteht jedoch darin, dass sie eine vollständige und genaue Umgebungsrepräsentation voraussetzen. Das bedeutet sobald Bauteile von ihrer erwarteten Lage abweichen, liefern sie keine Korrekturmechanismen und scheitern [4].

Ein praxisnäherer Ansatz stellt das sogenannte Drag-to-Teach dar: Eine menschliche Fachkraft führt den Roboterarm manuell durch eine Aufgabe und die aufgezeichnete Trajektorie wird anschließend wiederholt [9]. Dieses Verfahren erfordert keinerlei Programmierkenntnisse, ist jedoch grundsätzlich auf exakt reproduzierbare Umgebungsbedingungen angewiesen, sodass Lageabweichungen unmittelbar zum Versagen der aufgezeichneten Bewegung führen [10].

Lernbasierte Verfahren setzen an dieser Grenze an. Reinforcement Learning (RL) ermöglicht es einem Agenten, durch wiederholte Interaktion mit einer Umgebung eine Strategie zu erlernen, die eine vorgegebene Belohnungsfunktion maximiert [5]. Herausforderungen bleiben jedoch der hohe Stichprobenbedarf, die aufwendige Gestaltung geeigneter Belohnungsfunktionen sowie der Transfer von der Simulation auf reale Systeme [5].

Imitation Learning (IL), auch Learning from Demonstration genannt, umgeht den Bedarf an expliziter Belohnungsdefinition, indem aus menschlichen Demonstrationen direkt eine Strategie abgeleitet wird [11]. Erreichte Ergebnisse zeigen, dass IL als Alternative mit geringerem Datenbedarf an expliziter Belohnung untersucht wurde. IL umgeht die Belohnungsdefinition, verlagert den Datenbedarf jedoch auf die Demonstrationen selbst [11].

Beiden Ansätzen gemeinsam ist das Problem des Sim-to-Real-Gaps: In der Simulation erlernte Strategien lassen sich aufgrund von Diskrepanzen in Dynamik, Reibung und Sensorverhalten häufig nicht direkt übertragen [6].

Die jüngste Entwicklung in der robotischen Manipulation sind sogenannte Foundation Models und Vision-Language Models (VLMs): große, auf umfangreichen Datensätzen vortrainierte Modelle, die Wahrnehmung und Sprachverständnis kombinieren und Aufgaben auf Basis natürlichsprachlicher Anweisungen ausführen können [12]. Ihr Potenzial liegt in der Generalisierung über Aufgaben und Objekte hinweg, ohne aufgabenspezifisches Neutraining [12]. Zum aktuellen Zeitpunkt befinden sich diese Ansätze noch im experimentellen Stadium: Latenz, Rechenanforderungen und Inferenzgeschwindigkeit schließen einen produktiven Einsatz unter realen Montagebedingungen derzeit aus; hinzu kommt die fehlende Integration in die Embedded-Controller kompakter, ressourcenbeschränkter Roboterarme [12].

Ein systematischer Vergleich dieser Ansätze (klassisch, lernbasiert und modellbasiert) speziell für kompakte Roboterarme mit Hardware-Einschränkungen unter realen Montagebedingungen ist nach aktuellem Kenntnisstand nicht umfassend dokumentiert. Bestehende Arbeiten evaluieren einzelne Verfahren überwiegend isoliert oder unter idealisierten Laborbedingungen; eine kriteriengeleitete Gegenüberstellung, die Adaptivität, Implementierungsaufwand und Sim-to-Real-Fähigkeit gleichzeitig berücksichtigt, liegt nach Kenntnis des Autors nicht vor. Diese Lücke motiviert den in Abschnitt 1.2 beschriebenen Ansatz dieser Vorarbeit.

## 1.4 Aufbau des Berichts

Der vorliegende Bericht gliedert sich in vier Kapitel. Kapitel 2 legt die theoretischen Grundlagen dar: Es führt den Dobot Magician als Hardwareplattform ein, beschreibt die verwendete Simulationsumgebung und definiert zentrale Begriffe wie Flexibilität im Kontext roboterbasierter Montage. Kapitel 3 stellt die betrachteten Verfahren – klassische Planungsansätze, lernbasierte Methoden sowie Foundation Models – systematisch vor und bewertet sie anhand definierter Kriterien in einer Bewertungsmatrix. Kapitel 4 fasst die Ergebnisse des Vergleichs zusammen und leitet daraus eine begründete Empfehlung für die vertiefte Untersuchung im Rahmen der Bachelorarbeit ab.

# 2 Grundlagen und theoretischer Rahmen

Dieses Kapitel beschreibt die technischen und theoretischen Grundlagen der vorliegenden Arbeit. Zunächst wird der verwendete Roboterarm Dobot Magician als experimentelle Plattform vorgestellt. Anschließend werden die eingesetzte Simulationsumgebung sowie die Definition von Flexibilität im Kontext der Arbeit erläutert. Abschließend erfolgt eine Einordnung von Montage- und Demontageaufgaben als Robotikproblem. Die dargestellten Grundlagen bilden die Basis für die Entwicklung und Bewertung der eingesetzten Lernverfahren.

## 2.1 Der Dobot Magician als Plattform

Im folgenden Abschnitt werden die technischen Spezifikationen des Roboterarms vorgestellt und beschrieben. Diese Informationen sind entscheidend, um die Anforderungen an die im folgenden Kapiteln behandelten Lernansätze zu verstehen und bewerten. Die Spezifikationen wurden aus dem offiziellen Datenblatt des Dobot Magician entnommen und in Bezug auf die Anforderungen der Montage- und Demontageaufgaben analysiert.

### 2.1.1 Grundaufbau

Der Dobot Magician ist ein kompakter, vierachsiger Roboterarm, der häufig für Bildungszwecke und Forschungsaufgaben eingesetzt wird.

Er verfügt über eine Tischmontage sowie eine integrierte Steuerung in der Basis. Der Arm besteht aus Aluminium und ABS-Kunststoff und besitzt ein Eigengewicht von 3,4 kg (ohne Werkzeug beziehungsweise Endeffektor) [13].

Die vier Achsen des Roboterarms sind wie folgt konfiguriert: J1 (Basisrotation), J2 (Schultergelenk), J3 (Ellenbogengelenk) sowie J4 (Handgelenk beziehungsweise Greiferrotation). Diese Konfiguration ermöglicht ein breites Bewegungspektrum, das für Montage- und Demontageaufgaben relevant sind[13].

### 2.1.2 Bewegung & Präzision

Die maximale Reichweite des Dobot Magician beträgt 320 mm [13] und definiert damit den Arbeitsraum des Benchmark-Tasks.

Die Lernansätze müssen in der Lage sein, innerhalb dieses Bereichs präzise zu agieren. Die Wiederholgenauigkeit liegt bei $\pm 0{,}2$ mm [13] und stellt damit die physikalische Obergrenze der erreichbaren sowie die untere Grenze der reproduzierbaren Positionsgenauigkeit des Systems dar.

Daraus ergibt sich die Anforderung, dass alle Ansätze diese Toleranz einhalten oder übertreffen müssen, um sicherzustellen, dass die Hardware nicht zum limitierenden Faktor wird. Die maximale Gelenkgeschwindigkeit beträgt 320°/s bei einer Nutzlast von 250 g, während das Rotationsservo bis zu 480°/s erreicht [13]. Diese dynamischen Grenzen bestimmen die Anforderungen an Stabilität und Reaktionsfähigkeit der eingesetzten Lernverfahren.

### 2.1.3 Nutzlast & Dynamik

Die maximale Traglast des Dobot Magician beträgt 500 g [13]. Bei einer Nutzlast von 250 g wird die Höchstgeschwindigkeit erreicht, während höhere Lasten die Dynamik des Systems reduzieren.

Diese Einschränkung begrenzt die Auswahl geeigneter Benchmark-Objekte. Montage- und Demontageobjekte müssen innerhalb der spezifizierten Traglast liegen, um einen stabilen und reproduzierbaren Betrieb des Systems sicherzustellen. Objekte mit einer Masse oberhalb von 500 g werden daher ausgeschlossen.

### 2.1.4 Schnittstellen & Kommunikation

Angesteuert wird der Dobot Magician standardmäßig über USB. Kabellose Kommunikation via WLAN oder Bluetooth ist ausschließlich bei Modellen mit „Advanced Communication"-Erweiterung verfügbar [13].

Die I/O-Schnittstelle an der Basis stellt zwei schaltbare 12-V- und zwei 5-V-Ausgänge sowie vier 3, 3-V-Ein-/Ausgänge bereit, von denen zwei als PWM-Ausgänge (Pulsweitenmodulation) und zwei als ADC-Eingänge (Analog-Digital-Converter) konfiguriert sind, zusätzlich sind drei RS232-Schnittstellen verfügbar [13]. Am Arm selbst stehen zwei schaltbare 12-V-Ausgänge sowie sieben 3,3-V-Ein-/Ausgänge zur Verfügung, davon drei PWM- und vier ADC-Kanäle [13].

Zur Steuerung wird standardmäßig die Software DOBOTStudio eingesetzt, die für Windows und macOS verfügbar ist [13]. Sie unterstützt grundlegende Programmierung und Bewegungssteuerung, bietet jedoch keine native Integration in Simulationsumgebungen oder Middleware-Frameworks wie ROS.

### 2.1.5 Betrieb & Umgebung

Der Dobot Magician ist für den Laborbetrieb konzipiert und entspricht der Schutzklasse IP20. Daher ist er ist ausschließlich in trockenen Innenräumen zu betreiben [13]. Die zulässige Umgebungstemperatur liegt zwischen $-10\,^{\circ}\mathrm{C}$ bis $60\,^{\circ}\mathrm{C}$ [13].

Die Stromversorgung erfolgt über 100–240 V AC, der intern auf 12 V DC bei maximal 7 A gewandelt wird. Die maximale Leistungsaufnahme beträgt 60 W [13].

## 2.2 Entwicklungs- und Simulationsumgebung

### 2.2.1 Virtuelle Maschine & Betriebssystem

Als Betriebssystem wird Ubuntu Server 22.04 LTS (Desktop-Version) für ARM eingesetzt, das in einer virtuellen Maschine per VMware Fusion auf einem MacBook Air mit Apple-M5-Prozessor betrieben wird. Die Wahl fiel auf diese Distribution, da sie sich aufgrund ihres umfangreichen Software-Ökosystems, der breiten Unterstützung gängiger Robotik-Frameworks wie ROS sowie der Kompatibilität mit Simulationsumgebungen wie MuJoCo als De-facto-Standard in der Robotikforschung etabliert hat [14]. Die Verwendung der Desktop-Variante gegenüber der Server-Edition begründet sich in den grafischen Anforderungen der eingesetzten Simulationsumgebungen, die eine visuelle Aus-

gabe voraussetzen. Der Betrieb als virtuelle Maschine auf Apple-Silicon-Hardware erfolgt nativ auf ARM64-Basis, wodurch eine hohe Kompatibilität mit modernen Robotik-Paketen gewährleistet ist, die zunehmend für diese Architektur bereitgestellt werden.

### 2.2.2 ROS2 & RViz

Bei ROS2 (Robot Operating System 2) handelt es sich um ein Open-Source-Middleware-Framework, das als Kommunikationsschicht zwischen Hardware, Sensoren und Softwarekomponenten in der Robotik dient. Entgegen seiner Bezeichnung ist ROS2 kein Betriebssystem im klassischen Sinne, sondern eine Abstraktionsschicht, die die Interaktion verschiedener Komponenten über sogenannte Topics, Services und Actions ermöglicht [15].

Die Kommunikation innerhalb ROS2 basiert auf dem Data Distribution Service (DDS), einem standardisierten Publish-Subscribe-Protokoll, das eine dezentrale, echtzeitfähige Datenkommunikation zwischen Knoten (Nodes) ermöglicht. Gegenüber seinem Vorgänger ROS1 bietet ROS2 damit verbesserte Echtzeiteigenschaften, eine robustere Multi-Roboter-Unterstützung sowie eine aktiv gepflegte Codebasis, da ROS1 im Jahr 2025 das offizielle End-of-Life erreichte [15].

ROS2 spielt in diesem Setup eine zentrale Rolle, da es die Kommunikation zwischen dem Dobot-Treiber, der MuJoCo-Simulation und der Steuerungslogik koordiniert. Über ROS2 werden Sensordaten, Befehle und Statusinformationen ausgetauscht, was eine nahtlose Integration der verschiedenen Komponenten ermöglicht.

RViz (ROS Visualization) ist ein 3D-Visualisierungstool für ROS, das die Darstellung von Roboterzuständen, Gelenkwinkeln, Koordinatensystemen (TF-Frames) sowie geplanten Trajektorien in Echtzeit ermöglicht [16]. Es dient hauptsächlich der Überprüfung von Kinematik, TF-Frames und geplanten Trajektorien vor der realen Ausführung, bietet jedoch keine physikalische Simulation, Kollisionsberechnung oder Lernumgebung [16]. RViz dient in dieser Arbeit somit primär als Visualisierungs- und Debugging-Tool, zur Verifikation kinematischer Zusammenhänge, der Überprüfung von TF-Frames sowie geplanten Bewegungspfaden vor der Ausführung auf realer Hardware.

### 2.2.3 MuJoCo als Lernumgebung

MuJoCo (Multi-Joint dynamics with Contact) ist eine physikbasierte Simulationsumgebung, die auf präzise Kontakt- und Reibungsmodellierung spezialisiert ist und von DeepMind entwickelt sowie als Open-Source-Projekt bereitgestellt wird [17].

Im Vergleich zu alternativen Simulationsumgebungen wie Gazebo, die primär auf vollständige Robotersystemsimulation ausgelegt sind, zeichnet sich MuJoCo durch eine höhere Recheneffizienz sowie eine für Reinforcement-Learning-Anwendungen optimierte Physik-Engine aus [17]. Insbesondere die Genauigkeit der Kontaktkraftmodellierung macht MuJoCo zur bevorzugten Umgebung für das Training von Policies in Greif- und Montageaufgaben, bei denen Reibung, Gelenkmechanik und Kontaktkräfte eine zentrale Rolle spielen.

Der Dobot Magician wird in MuJoCo über eine URDF-Modelldatei abgebildet, die

Gelenke, Massen und Kollisionsgeometrien des Arms beschreibt. Das verwendete Modell stammt aus einem internen GitLab-Repository der HFT Stuttgart [18].

MuJoCo wird in dieser Arbeit in Kombination mit Gymnasium, dem standardisierten Nachfolger der OpenAI-Gym-Bibliothek, verwendet [19]. Gymnasium stellt eine einheitliche Schnittstelle für RL-Umgebungen bereit, über die Aktionsraum, Beobachtungsraum sowie der Interaktionszyklus zwischen Agent und Umgebung (Observation, Action, Reward, Terminated) definiert werden. Diese Standardisierung ermöglicht die weitgehend modulare Austauschbarkeit von RL-Algorithmen und Simulationsumgebungen.

Ein wesentlicher Vorteil von MuJoCo für RL-basierte Ansätze liegt in der Möglichkeit, mehrere Simulationsinstanzen parallel auszuführen, wodurch ein erheblich beschleunigtes Training ohne Verschleiß physischer Hardware realisiert werden kann [20]. Die Qualität des Sim-to-Real-Transfers hängt dabei maßgeblich davon ab, wie präzise die Simulation physikalische Eigenschaften wie Reibung, Trägheit und Kontaktkräfte abbildet, je realistischer das Simulationsmodell, desto robuster die Übertragung der trainierten Policy auf den echten Dobot Magician.

### 2.2.4 Konzept des Sim-to-Real-Transfer

Der Sim-to-Real-Transfer bezeichnet die Übertragung einer in einer Simulationsumgebung trainierten Policy auf reale Hardware. Die Simulation dient dabei als kostengünstiger und risikofreier Trainingsraum, in dem eine nahezu unbegrenzte Anzahl von Trainingsepisoden ohne Verschleiß oder Sicherheitsrisiken durchgeführt werden kann [6].

In der Praxis zeigt sich jedoch, dass in der Simulation erfolgreich trainierte Policies auf realer Hardware häufig deutlich schlechter abschneiden oder gänzlich versagen. Ursache hierfür ist die sogenannte *Reality Gap*, also die Gesamtheit aller Unterschiede zwischen dem Simulationsmodell und der realen Welt [21]. Diese umfasst unter anderem ungenaue Massenverteilungen, nicht modellierte Gelenkreibung, Kameraverzeichnung sowie systembedingte Latenzen. Da kein Simulationsmodell die Realität vollständig abbilden kann, ist die Reality Gap ein fundamentales Problem des simulationsbasierten Lernens.

Zur Überbrückung dieser Lücke haben sich in der Literatur mehrere Strategien etabliert [22]:*Domain Randomization* variiert Simulationsparameter wie Masse, Reibungskoeffizienten, Beleuchtung und Texturen während des Trainings zufällig, sodass die Policy eine inhärente Robustheit gegenüber Parameterabweichungen entwickelt. *Domain Adaptation* verfolgt den komplementären Ansatz, Simulation und Realität durch gezielte Kalibrierung des Simulationsmodells anhand realer Messdaten anzunähern [23]. Darüber hinaus existieren Methoden wie *System Identification*, bei der reale physikalische Parameter systematisch gemessen und ins Modell eingespeist werden, sowie *Residual Learning*, das einen klassischen Regler mit einem lernbasierten Korrekturterm kombiniert, um Modellungenauigkeiten zur Laufzeit zu kompensieren [24].

Für Greif- und Montageaufgaben ist der Sim-to-Real-Transfer besonders anspruchsvoll, da diese Aufgaben in hohem Maße kontaktsensitiv sind. Bereits kleine Fehler in der Reibungsmodellierung können zu grundlegend abweichendem Greifverhalten in der Realität führen. Somit ist dies ein Aspekt, der die Wahl von MuJoCo als Simulati-

onsumgebung mit präziser Kontaktmodellierung zusätzlich begründet (vgl. Abschnitt 2.2.3).

Ein zusätzlicher, bei VLM-basierten Ansätzen besonders kritischer Aspekt ist der visuelle Sim-to-Real-Gap: Während Simulationen idealisierte, rauschfreie Bilder erzeugen, liefert das eingesetzte Vision-Kit in der Realität Bilder mit begrenzter Auflösung, Sensorrauschen und variierenden Lichtverhältnissen. Diese Diskrepanz kann die Erkennungsleistung von Vision-Language-Modellen wie OpenVLA erheblich beeinträchtigen [25].

Da alle in dieser Arbeit untersuchten Lernansätze (Reinforcement Learning, Imitation Learning sowie VLM-basierte Steuerung) zumindest teilweise in MuJoCo trainiert werden, stellt die Übertragbarkeit auf den echten Dobot Magician ein durchgängiges Bewertungskriterium dar und wird in der vergleichenden Analyse in Kapitel 3 explizit berücksichtigt.

## 2.3 Definition von Flexibilität im Kontext dieser Arbeit

Der Begriff „Flexibilität“ wird in der Robotik nicht einheitlich verwendet [26] und bedarf daher im Kontext dieser Arbeit einer klaren Definition. Grundsätzlich lassen sich zwei Verständnisse unterscheiden: Zum einen die vollständige Generalisierung, also die Fähigkeit eines Systems, mit beliebigen unbekannten Objekten oder Aufgaben umzugehen [27]. Zum anderen die aufgabenbezogene Robustheit, die Fähigkeit, innerhalb eines bekannten Aufgabenraums trotz kleiner Abweichungen zuverlässig zu funktionieren [28].

Im Rahmen dieser Arbeit wird Flexibilität im zweiten Sinne verstanden. Ein System gilt als flexibel, wenn es bekannte Bauteile in den folgenden drei Szenarien korrekt greifen und montieren kann, ohne manuelle Neuprogrammierung: (1) bei geometrischen Lageabweichungen, (2) bei veränderten visuellen Eigenschaften wie abweichender Farbe sowie (3) bei strukturellen Veränderungen wie Beschädigungen.

Die geometrische Flexibilität, adressiert die erste und in der Praxis am häufigsten auftretende Dimension [29]: Ein Bauteil liegt 5–15 mm versetzt oder leicht verdreht auf der Arbeitsfläche. Der Roboter soll die Abweichung selbstständig erkennen und seine Greifposition entsprechend anpassen, ohne dass ein Bediener eingreifen oder die Trajektorie neu programmieren muss. Diese Dimension ist vergleichsweise gut handhabbar, da klassische Bildverarbeitung in vielen Fällen ausreicht, um Versatz oder Verdrehung zu detektieren und die Greifposition entsprechend anzupassen.

Die zweite Dimension betrifft visuelle Veränderungen, konkret Farbabweichungen. Klassische Bildverarbeitungsansätze arbeiten häufig mit festen Farbschwellen [30] und würden bei einem andersfarbigen Bauteil nicht zuverlässig funktionieren [31].Ein flexibles System muss das Objekt stattdessen form- oder semantikbasiert identifizieren. Dies erfordert bereits ein tieferes, semantisches Verständnis des Objekts, da rein farbbasierte Erkennungsansätze hier versagen.

Die dritte und anspruchsvollste Dimension ist strukturelle Flexibilität: Das Bauteil ist beschädigt, etwa verrostet, verformt oder verschmutzt. Hier reicht es nicht mehr aus, eine angepasste Greifposition zu berechnen, sondern das System muss beurteilen, ob ein Greifen überhaupt noch möglich ist, und eine definierte Fehlerbehandlung einleiten wie zum Beispiel das Bauteil als nicht greifbar klassifizieren und einen Fehler signalisieren.

Dies ist die anspruchsvollste Dimension, da sich die Objektgeometrie selbst verändert und damit die Grundlage klassischer wie auch lernbasierter Erkennungsverfahren direkt betroffen ist.

Diese drei Dimensionen orientieren sich an realen industriellen Anforderungen. Sie spiegeln Variationen wider, die in realen industriellen Montageprozessen regelmäßig auftreten [32]: Bauteile kommen selten exakt positioniert, immer gleich gefärbt und unbeschädigt an. Ein System, das nur unter Idealbedingungen funktioniert, ist für den praktischen Einsatz nicht ausreichend.

Die vollständige Generalisierung auf beliebige, unbekannte Objekte oder Aufgaben liegt damit bewusst außerhalb des Rahmens dieser Arbeit. Dies begründet sich zum einen in den in Abschnitt 2.1.3 beschriebenen hardwareseitigen Grenzen des Dobot Magician, insbesondere hinsichtlich Nutzlast und Arbeitsraum, zum anderen darin, dass die drei definierten Dimensionen bereits einen für den Anwendungskontext praxisrelevanten Flexibilitätsbegriff abdecken.

In Kapitel 3 dienen die drei Dimensionen als durchgängiges Bewertungskriterium für die untersuchten Lernansätze. Es wird explizit analysiert, inwieweit die verschiedenen Ansätze in der Lage sind, mit den jeweiligen Herausforderungen umzugehen und welche Grenzen sich dabei zeigen.

## 2.4 Montage und Demontage als Robotikproblem

Greifplanung (Grasp Planning) ist die algorithmische Berechnung einer stabilen Greifkonfiguration für den Roboter-Greifer. Sie bestimmt präzise die Kontaktpunkte, die Greifgeometrie sowie die Greifkräfte und -Momente, mit denen das Objekt gefasst werden soll. Ziel ist ein sicheres Halten und Manipulieren, das ein Abrutschen verhindert, oft unter Nutzung von Konzepten wie Formschluss oder Kraftschluss. Dies berücksichtigt Objektgeometrie, Oberflächenbeschaffenheit und die geplante Manipulationsaufgabe.

Da die optimale Greifposition von Objektgeometrie, Masse, Schwerpunkt, Oberflächenbeschaffenheit und der nachfolgenden Aufgabe abhängt und all das gleichzeitig berücksichtigt werden muss, ist die Greifplanung nicht trivial. Es gibt analytische Methoden und lernbasierte Methoden, die aus Daten stabile Greifpositionen ableiten [33]. Bei Montageaufgaben ist Greifplanung besonders anspruchsvoll, weil das Bauteil nicht nur stabil gegriffen, sondern auch präzise positioniert und orientiert werden muss [34]. Zum Beispiel muss der Griff die nachfolgende Fügebewegung ermöglichen, ohne das Bauteil neu ausrichten zu müssen.

Die Fähigkeit des Systems, den Moment und die Art des physischen Kontakts zwischen Greifer und Objekt zu erkennen wird in der Robotik als Kontakterkennung bezeichnet. Dies erfolgt typischerweise über dedizierte Kraft- oder taktile Sensoren oder indirekt mittels Methoden wie der Strommessung der Motoren. Bei Montageaufgaben ist Kontakterkennung besonders kritisch, da kontrolliertes Fügen gefordert ist: Ein zu früher Kontakt birgt das Risiko von Kollisionen, ein zu später erfolgender Kontakt das Risiko einer Fehlpositionierung. Ohne zuverlässige Kontakterkennung bleibt das Fügen blind und fehleranfällig. Herausfordernd ist deshalb, dass der Dobot Magician über keine dedizierten Kraft- oder Taktilsensoren verfügt, siehe Abschnitt 2.1. Kontakt

kann lediglich indirekt über erhöhte Motorströme oder positionsbasierte Abweichungen erkannt werden, was die Kontakterkennung erheblich erschwert. Als Alternativen zu direkter Kontakterkennung kommen Impedanzregelung, nachgiebige Greiferdesigns (compliant grippers) oder visuelle Schätzung des Kontakts anhand von Kameradaten infrage.

Kraft-Momenten-Steuerung bezeichnet eine Regelungsstrategie, bei der nicht nur die Position des Endeffektors, sondern auch die auf den Roboter wirkenden Kräfte und Momente aktiv geregelt werden. Der Roboter erfasst dabei den mechanischen Widerstand seiner Umgebung und passt seine Bewegung entsprechend an. Bei Montageaufgaben ist diese Fähigkeit grundlegend notwendig, da Fügeoperationen kontrollierte Kontaktkräfte erfordern: Zu hohe Kräfte beschädigen das Bauteil oder die Aufnahme, während zu geringe Kräfte ein vollständiges Fügen verhindern. Als etablierte Regelungsarten sind die Impedanzregelung (bei der Steifigkeit und Dämpfung vorgegeben werden), die Admittanzregelung (bei der Kraft als Eingang und Bewegung als Ausgang dient) sowie hybride Kraft-Position-Regelungsansätze zu nennen [35]. Der Dobot Magician verfügt jedoch über keine integrierte Kraft-Momenten-Sensorik (siehe Abschnitt 2.1), weshalb eine echte Kraft-Momenten-Steuerung ohne externe Sensorerweiterung nicht realisierbar ist. Dies stellt eine wesentliche Einschränkung bei Montageaufgaben mit engen Toleranzen dar.

Kollisionsvermeidung bei engen Toleranzen bedeutet im Kontext von Montageaufgaben, dass der Roboter Bauteile präzise in enge Aufnahmen fügen muss, ohne dabei angrenzende Strukturen zu berühren. Bei Toleranzen im Millimeterbereich stellt dies eine erhebliche Herausforderung dar. Klassische Methoden umfassen geometriebasierte Planungsverfahren wie RRT (Rapidly-exploring Random Trees) [36] oder PRM (Probabilistic Roadmap Methods) [37], sowie potentialfeldbasierte Ansätze. Daneben existieren lernbasierte Ansätze, die Kollisionsvermeidung implizit aus Demonstrationen ableiten. Letztere sind bei engen Toleranzen jedoch problematisch, da lernbasierte Methoden grundsätzlich generalisieren und dabei Ungenauigkeiten entstehen können (siehe Abschnitt 2.2.4), die klassische Planungsalgorithmen durch exakte geometrische Modellierung vermeiden.

Die Demontage stellt gegenüber der Montage eine grundlegend schwierigere Aufgabe dar. Während bei der Montage der Ausgangszustand bekannt und kontrollierbar ist, sind bei der Demontage die Verbindungszustände häufig unbekannt: Schrauben können festgerostet, Clips verformt oder Bauteile verklebt sein. Konkret bedeutet dies, dass der Roboter weder weiß, ob eine Verbindung überhaupt lösbar ist, noch wie viel Kraft dafür aufgewendet werden muss oder ob das Bauteil nach dem Lösen frei beweglich oder noch anderweitig fixiert ist. Verbindungen wie Clips, Presspassungen oder korrodierte Schrauben erfordern oft deutlich mehr Lösekraft als die ursprüngliche Fügekraft, wobei diese Kraft vorab in der Regel nicht bekannt ist. Unkontrollierte Demontage birgt daher das Risiko, Bauteile oder angrenzende Komponenten zu beschädigen, was besonders bei Recycling- oder Reparaturanwendungen kritisch ist, in denen Bauteile wiederverwendet werden sollen.

Lernbasierte Systeme könnten diesem Problem durch taktiles Feedback, Kraftprofilanalyse (anhand charakteristischer Kraft-Weg-Kurven beim Lösen) [38] begegnen oder durch eine Klassifikation des Verbindungstyps vor der Demontage begegnen.

Für VLM-basierte Ansätze ist die Demontage besonders interessant, da VLMs potenziell in der Lage sind, den Verbindungstyp direkt aus dem Kamerabild zu identifizieren und die Demontagestrategie semantisch abzuleiten beispielsweise indem der Verbindungstyp identifiziert und die Lösungsrichtung daraus ermittelt wird. Im industriellen Kontext gewinnt automatisierte Demontage durch Kreislaufwirtschaft, Recyclingpflichten und Reparierbarkeitsanforderungen (etwa die EU Right to Repair-Regulierung) [39] zunehmend an Bedeutung und ist ein aktives Forschungsfeld.

# 3 Fazit und begründete Empfehlung

# Literaturverzeichnis


[1] H. Bley, G. Reinhart, G. Seliger, M. Bernardi und T. Korne, „Appropriate Human Involvement in Assembly and Disassembly," *CIRP Annals*, Jg. 53, S. 487–509, 2004. DOI: 10.1016/s0007-8506(07)60026-2.

[2] E. Ouda und M. Haggag, „Automation in Modular Construction Manufacturing: A Comparative Analysis of Assembly Processes," *Sustainability*, 2024. DOI: 10.3390/su16219238.

[3] R. A. Adebayo, N. C. Obiuto, I. C. Festus-Ikhuoria und O. K. Olajiga, „Robotics in Manufacturing: A Review of Advances in Automation and Workforce Implications," *International Journal of Advanced Multidisciplinary Research and Studies*, 2024. DOI: 10.62225/2583049x.2024.4.2.2549.

[4] J. Su, L. Shen, Z. Peng und X. Qu, „Design of a robot system for reorienting and assembling irregular parts," *International Journal of Advanced Robotic Systems*, Jg. 21, 2024. DOI: 10.1177/17298806241229271.

[5] A. A. Shahid, D. Piga, F. Braghin und L. Roveda, „Continuous control actions learning and adaptation for robotic manipulation through reinforcement learning," *Autonomous Robots*, Jg. 46, S. 483–498, 2022. DOI: 10.1007/s10514-022-10034-z.

[6] E. Aljalbout, J. Xing, A. Romero u. a., „The Reality Gap in Robotics: Challenges, Solutions, and Best Practices," *ArXiv*, Jg. abs/2510.20808, 2025. DOI: 10.48550/arxiv.2510.20808.

[7] C. Ecker, „Interaktionsbasiertes Verfahren zur holistischen Konfiguration roboterbasierter Montagesysteme," Dissertation, Diss., RWTH Aachen University, Aachen, Deutschland, 2023. DOI: 10.18154/RWTH-2024-00164. Adresse: https://publications.rwth-aachen.de/record/975208.

[8] S. Karaman und E. Frazzoli, „Sampling-based algorithms for optimal motion planning," *The International Journal of Robotics Research*, Jg. 30, S. 846–894, 2011. DOI: 10.1177/0278364911406761.

[9] D. Zhu, Y. Gan und X. Dai, „Research on Drag Control Algorithm of Cooperative Robot Based on Torque Compensation," in *2021 China Automation Congress (CAC)*, 2021, S. 708–715. DOI: 10.1109/CAC53003.2021.9727597.

[10] M. Shneier und R. Bostelman, „Literature Review of Mobile Robots for Manufacturing," National Institute of Standards und Technology (NIST), Gaithersburg, MD, NIST Interagency/Internal Report (NISTIR) 8022, 2015. DOI: 10.6028/NIST.IR.8022. Adresse: https://nvlpubs.nist.gov/nistpubs/ir/2015/NIST.IR.8022.pdf.

[11] S. Reddy, A. Dragan und S. Levine, „SQIL: Imitation Learning via Reinforcement Learning with Sparse Rewards," *arXiv: Learning*, 2019. DOI: 10.48550/arxiv.1910.07224.

[12] M. J. Kim, K. Pertsch, S. Karamcheti u. a., *OpenVLA: An Open-Source Vision-Language-Action Model*, 2024. arXiv: 2406.09246 [cs.RO]. Adresse: https://arxiv.org/abs/2406.09246.

[13] Variobotic. „DOBOT Magician Der Desktoproboter für Education-, Pro- und Business-Anwender." (2026), Adresse: https://cdn-reichelt.de/documents/datenblatt/E400/DOBOT_MAGICIAN_DB-DE.pdf (besucht am 07. 05. 2026).

[14] M. Quigley, K. Conley, B. Gerkey u. a., „ROS: an open-source Robot Operating System," in *ICRA workshop on open source software*, Bd. 3, 2009, S. 5.

[15] S. Macenski, T. Foote, B. Gerkey, C. Lalancette und W. Woodall, „Robot Operating System 2: Design, architecture, and uses in the wild," *Science Robotics*, Jg. 7, Nr. 66, Mai 2022, ISSN: 2470-9476. DOI: 10.1126/scirobotics.abm6074. Adresse: http://dx.doi.org/10.1126/scirobotics.abm6074.

[16] Open Robotics. „RViz2 User Guide." Abgerufen am: 12.05.2026. (2026), Adresse: https://docs.ros.org/en/rolling/Concepts/Intermediate/About-Rviz.html.

[17] E. Todorov, T. Erez und Y. Tassa, „MuJoCo: A physics engine for model-based control," Okt. 2012, S. 5026–5033, ISBN: 978-1-4673-1737-5. DOI: 10.1109/IROS.2012.6386109.

[18] @. Felix Baechle, *dobot_mujoco*, https://gitlab.rz.hft-stuttgart.de/baechle/dobot_mujoco, Abgerufen am: 12.05.2026, 2026.

[19] G. Brockman, V. Cheung, L. Pettersson u. a., „OpenAI Gym," *arXiv preprint arXiv:1606.01540*, 2016.

[20] Y. Tassa, Y. Doron, A. Muldal u. a., „DeepMind Control Suite," *arXiv preprint arXiv:1801.00690*, 2018.

[21] J.-B. Mouret, S. Koos und S. Doncieux, *Crossing the Reality Gap: a Short Introduction to the Transferability Approach*, 2013. arXiv: 1307.1870 [cs.RO]. Adresse: https://arxiv.org/abs/1307.1870.

[22] J. Tobin, R. Fong, A. Ray, J. Schneider, W. Zaremba und P. Abbeel, *Domain Randomization for Transferring Deep Neural Networks from Simulation to the Real World*, 2017. arXiv: 1703.06907 [cs.RO]. Adresse: https://arxiv.org/abs/1703.06907.

[23] K. Bousmalis, A. Irpan, P. Wohlhart u. a., *Using Simulation and Domain Adaptation to Improve Efficiency of Deep Robotic Grasping*, 2017. arXiv: 1709.07857 [cs.LG]. Adresse: https://arxiv.org/abs/1709.07857.

[24] T. Johannink, S. Bahl, A. Nair u. a., *Residual Reinforcement Learning for Robot Control*, 2018. arXiv: 1812.03201 [cs.RO]. Adresse: https://arxiv.org/abs/1812.03201.

[25] K. Singh, T. Navaratnam, J. Holmer, S. Schaub-Meyer und S. Roth, *Is Synthetic Data all We Need? Benchmarking the Robustness of Models Trained with Synthetic Images*, 2024. arXiv: 2405.20469 [cs.CV]. Adresse: https://arxiv.org/abs/2405.20469.

[26] A. K. Sethi und S. P. Sethi, „Flexibility in manufacturing: a survey,“ *International Journal of Flexible Manufacturing Systems*, Jg. 2, Nr. 4, S. 289–328, 1990. Adresse: https://ssrn.com/abstract=1080305.

[27] O. Kroemer, S. Niekum und G. Konidaris, *A Review of Robot Learning for Manipulation: Challenges, Representations, and Algorithms*, 2020. arXiv: 1907.03146 [cs.RO]. Adresse: https://arxiv.org/abs/1907.03146.

[28] A. Billard und D. Kragic, „Trends and challenges in robot manipulation,“ *Science*, Jg. 364, Nr. 6446, eaat8414, 2019. DOI: 10.1126/science.aat8414.

[29] Z. Bi und W. Zhang, „Flexible fixture design and automation: Review, issues and future directions,“ *International Journal of Production Research*, Jg. 39, S. 2867–2894, Nov. 2010. DOI: 10.1080/00207540110054579.

[30] R. C. Gonzalez und R. E. Woods, *Digital Image Processing*. Pearson, 2018.

[31] R. Szeliski, *Computer Vision: Algorithms and Applications* (Texts in Computer Science), 2. Aufl. Cham: Springer Nature Switzerland, 2022, ISBN: 978-3-030-34371-2. DOI: 10.1007/978-3-030-34372-9.

[32] T. Bauernhansl, M. Hompel und B. Vogel-Heuser, *Industrie 4.0 in Produktion, Automatisierung und Logistik: Anwendung · Technologien · Migration*. Jan. 2014, ISBN: 978-3-658-04681-1. DOI: 10.1007/978-3-658-04682-8.

[33] J. Bohg, A. Morales, T. Asfour und D. Kragic, „Data-driven grasp synthesis—A survey,“ *IEEE Transactions on Robotics*, Jg. 30, Nr. 2, S. 289–309, 2014.

[34] R. Suárez, M. A. Roa und J. Cornella, „Grasp planning: Fundamentals and state-of-the-art,“ *UPC Research Reports*, 2006.

[35] B. Siciliano, L. Sciavicco, L. Villani und G. Oriolo, *Robotics: Modelling, Planning and Control*. Springer Science & Business Media, 2009.

[36] S. M. LaValle, „Rapidly-exploring random trees: A new tool for path planning,“ Iowa State University, Techn. Ber., 1998.

[37] L. E. Kavraki, P. Svestka, J.-C. Latombe und M. H. Overmars, „Probabilistic roadmaps for path planning in high-dimensional configuration spaces,“ *IEEE Transactions on Robotics and Automation*, Jg. 12, Nr. 4, S. 566–580, 1996.

[38] D. Sliwowski, S. Jadav, S. Stanovcic, J. Orbik, J. Heidersberger und D. Lee, „REASSEMBLE: A Multimodal Dataset for Contact-rich Robotic Assembly and Disassembly,“ 2025. arXiv: 2502.05086 [cs.RO]. Adresse: https://arxiv.org/abs/2502.05086.

[39] European Union, *Directive (EU) 2024/1799 of the European Parliament and of the Council of 13 June 2024 on common rules promoting the repair of goods*, Official Journal of the European Union, 2024. Adresse: https://eur-lex.europa.eu/eli/dir/2024/1799/oj.